Student 14 **Jeyakannan L** 192525376RD 2 / 2 submitted

Q1. Smart Hospital Patient Registration System

CODE

```
import java.util.*;
class Main
{
    public static String readInput(Scanner sc)
    {
        String s="";
        while(sc.hasNextLine())
        {
            s=sc.nextLine().trim();
            if(!s.isEmpty())
                return s;
        }
        return s;
    }
    public static String properCase(String str)
    {
        str=str.trim().toLowerCase();
        String words[]=str.split("\\s+");
        String ans="";
        for(int i=0;i<words.length;i++)
        {
            ans+=Character.toUpperCase(words[i].charAt(0));
            if(words[i].length()>1)
                ans+=words[i].substring(1);
            if(i!=words.length-1)
                ans+=" ";
        }
        return ans;
    }
    public static void main(String args[])
    {
        Scanner sc=new Scanner(System.in);
        ArrayList<Patient> patientList=new ArrayList<Patient>();
        LinkedHashMap<String,Patient> patientMap=new LinkedHashMap<String,Patient>();
        while(true)
        {
            int choice=Integer.parseInt(readInput(sc));
            if(choice==1)
            {
                String id=readInput(sc);
                String name=properCase(readInput(sc));
                String mobile=readInput(sc);
                String department=readInput(sc).toUpperCase();
                String doctor=readInput(sc);
                if(patientMap.containsKey(id))
                {
                    System.out.println("Duplicate Patient ID");
                    System.out.println();
                    continue;
                }
                if(!mobile.matches("\\d{10}"))
                {
                    System.out.println("Invalid Mobile Number");
                    System.out.println();
                    continue;
                }
                Patient p=new Patient(id,name,mobile,department,doctor);
                patientList.add(p);
                patientMap.put(id,p);
                System.out.println("Patient Registered Successfully.");
                System.out.println();
            }
```

```
            else if(choice==2)
            {
                String search=readInput(sc);
                if(search.equalsIgnoreCase("Search Patient:"))
                {
                    search=readInput(sc);
                }
                if(patientMap.containsKey(search))
                {
                    Patient p=patientMap.get(search);
                    System.out.println("Patient Details");
                    System.out.println();
                    System.out.println("ID : "+p.getId());
                    System.out.println();
                    System.out.println("Name : "+p.getName());
                    System.out.println();
                    System.out.println("Mobile : "+p.getMobile());
                    System.out.println();
                    System.out.println("Department : "+p.getDepartment());
                    System.out.println();
                    System.out.println("Doctor : "+p.getDoctor());
                }
                else
                {
                    System.out.println("Patient Not Found");
                }
            }
            else if(choice==3)
            {
                System.out.println("---------------------------------");
                System.out.println("ID Name Department");
                System.out.println("---------------------------------");
                for(Patient p:patientList)
                {
                    System.out.println(p.getId()+" "+p.getName()+" "+p.getDepartment());
                }
                System.out.println();
}
            else if(choice==4)
            {
                LinkedHashMap<String,Integer> departmentCount=new LinkedHashMap<String,Integer>();
                for(Patient p:patientList)
                {
                    String dept=p.getDepartment();
                    if(departmentCount.containsKey(dept))
                    {
                        departmentCount.put(dept,departmentCount.get(dept)+1);
                    }
                    else
                    {
                        departmentCount.put(dept,1);
                    }
                }
                System.out.println("Department Count");
                System.out.println();
                for(String dept:departmentCount.keySet())
                {
                    System.out.println(dept+" : "+departmentCount.get(dept));
                    System.out.println();
                }
            }
            else if(choice==5)
            {
                break;
            }
```

```
        }
    }
}
class Patient
{
    private String id;
    private String name;
    private String mobile;
    private String department;
    private String doctor;
    public Patient()
    {
    }
    public Patient(String id,String name,String mobile,String department,String doctor)
    {
        this.id=id;
        this.name=name;
        this.mobile=mobile;
        this.department=department;
        this.doctor=doctor;
    }
    public String getId()
    {
        return id;
    }
    public String getName()
    {
        return name;
    }
    public String getMobile()
    {
        return mobile;
    }
    public String getDepartment()
    {
        return department;
    }
    public String getDoctor()
    {
        return doctor;
    }
    public void setId(String id)
    {
        this.id=id;
    }
    public void setName(String name)
    {
        this.name=name;
    }
    public void setMobile(String mobile)
    {
        this.mobile=mobile;
    }
    public void setDepartment(String department)
    {
        this.department=department;
    }
    public void setDoctor(String doctor)
    {
        this.doctor=doctor;
    }

}
```

INPUT

```
1
P101
Rahul Kumar
9876543210
Cardiology
Dr.Arun
1
P102
Priya
9876543211
Ortho
Dr. Meena
3
4
2
P101
5
```

OUTPUT

```
(no output)
```

Q2. Online Library Book Management System

CODE

```
import java.util.Scanner;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
public class Main {
static class Book {
String id;
String title;
String author;
String category;
String publisher;
Book(String id, String title, String author, String category, String publisher) {
this.id = id;
this.title = title;
this.author = author;
this.category = category;
this.publisher = publisher;
}
}
static String trimSafe(String s) {
if (s == null) return "";
return s.trim();
}
static String toTitleCase(String s) {
s = trimSafe(s);
if (s.length() == 0) return s;
s = s.toLowerCase();
StringBuilder out = new StringBuilder();
boolean newWord = true;
for (int i = 0; i < s.length(); i++) {
char ch = s.charAt(i);
if (ch == ' ') {
out.append(' ');
newWord = true;
} else {
if (newWord) {
if (ch >= 'a' && ch <= 'z') ch = (char) (ch - 'a' + 'A');
newWord = false;
}
out.append(ch);
}
}
return out.toString().replaceAll(" +", " ");
}
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
ArrayList<Book> books = new ArrayList<>();
HashMap<String, Book> bookById = new HashMap<>();
HashSet<String> authors = new HashSet<>();
int maxBooks = 500;
while (true) {
if (!sc.hasNextLine()) break;
String line = sc.nextLine().trim();
if (line.length() == 0) continue;
int choice;
try {
choice = Integer.parseInt(line);
} catch (Exception e) {
continue;
}
if (choice == 6) break;
if (choice == 1) {
```

```
if (books.size() >= maxBooks) continue;
String id = trimSafe(sc.nextLine());
String title = trimSafe(sc.nextLine());
String author = trimSafe(sc.nextLine());
String category = trimSafe(sc.nextLine());
String publisher = trimSafe(sc.nextLine());
boolean ok = true;
if (id.length() == 0) ok = false;
if (title.length() == 0) ok = false;
if (author.length() == 0) ok = false;
if (category.length() == 0) ok = false;
if (publisher.length() == 0) ok = false;
if (bookById.containsKey(id)) ok = false;
if (!ok) continue;
title = toTitleCase(title);
category = category.toUpperCase();
Book b = new Book(id, title, author, category, publisher);
books.add(b);
bookById.put(id, b);
authors.add(author);
System.out.println("Book Added Successfully.");
} else if (choice == 2) {
String id = trimSafe(sc.nextLine());
Book b = bookById.get(id);
if (b == null) {
System.out.println("Book Not Available");
} else {
System.out.println("Book ID : " + b.id);
System.out.println("Book Title : " + b.title);
System.out.println("Author : " + b.author);
System.out.println("Category : " + b.category);
System.out.println("Publisher : " + b.publisher);
}
} else if (choice == 3) {
System.out.println("------------------------------");
System.out.println("ID TITLE AUTHOR CATEGORY");
System.out.println("------------------------------");
for (int i = 0; i < books.size(); i++) {
Book b = books.get(i);
System.out.println(b.id + " " + b.title + " " + b.author + " " + b.category);
}
} else if (choice == 4) {
System.out.println("Unique Authors");
for (String a : authors) {
System.out.println(a);
}
} else if (choice == 5) {
HashMap<String, Integer> catCount = new HashMap<>();
for (int i = 0; i < books.size(); i++) {
Book b = books.get(i);
String c = b.category;
Integer old = catCount.get(c);
if (old == null) catCount.put(c, 1);
else catCount.put(c, old + 1);
}
System.out.println("Category-wise Count");
for (String key : catCount.keySet()) {
System.out.println(key + " : " + catCount.get(key));
}
for (int i = 0; i < books.size(); i++) {
Book b = books.get(i);
System.out.println("Book Details");
System.out.println("Book ID : " + b.id);
System.out.println("Title : " + b.title);
System.out.println("Author : " + b.author);
```

```
System.out.println("Category : " + b.category);
System.out.println("Publisher : " + b.publisher);
String t = b.title.trim();
int len = b.title.length();
int words = 0;
if (t.length() > 0) {
words = 1;
for (int k = 0; k < t.length(); k++) {
if (t.charAt(k) == ' ') words++;
}
}
System.out.println("Length : " + len);
System.out.println("Words : " + words);
System.out.println("Category : " + b.category);
}
}
}
sc.close();
}
}
```

INPUT

```
1
1
B101
Java Programming Language
Herbert Schildt
Programming
McGraw Hill
1
B102
Data Structures
Mark Allen
Computer Science
Pearson
2
B101
3
4
5
6
Java Programming Language
Herbert Schildt
Programming
McGraw Hill
1
B102
Data Structures
Mark Allen
Computer Science
Pearson
2
B101
3
4
5
```

OUTPUT

```
(no output)
```